

Week 7 | 课时 3 | 从 section 到 chunk：结构感知切片与 overlap 边界

本课导航

先尊重文档结构，再处理长度	1
这节课解决什么问题	1
先用大白话讲	2
核心原理：chunk 是 Week8 的索引候选单元	2
参考学习时间	2
学完这一讲，你应该能做到什么	2
本课产出	2
fixed-size vs section-aware	2
场景切法决策表	3
1. section_aware_v1 的默认规则	3
2. table-aware 与 page-aware	3
3. chunk ID 必须稳定	3
4. overlap 的边界	4
5. chunk metadata 继承规则	4
6. chunk 合格的最低标准	5
7. 坏案例	5
企业最佳实践	5
课堂练习	5
本课最重要判断	6
自检清单	6
最小行动命令	6
延伸阅读	6
Footnotes	6

先尊重文档结构，再处理长度

这一讲处理 Week7 最容易被低估的环节：

chunking 不是把文本切成差不多长，而是把 section、表格、页码和上下文边界转成可索引单元。

这节课解决什么问题

固定长度切片看起来简单，但它会制造很多隐性事故：

- 把标题和正文切开
- 把步骤 1、2、3 切到不同 chunk
- 把表头和数据行拆散
- 把错误码和处理建议拆开
- overlap 过大导致重复噪声

Week7 的默认策略是 [section_aware_v1](#)：

heading-aware + section-aware + table-aware + page-aware + token-aware。

先用大白话讲

Chunking 可以理解成“把一本手册切成可检索证据卡”。卡片太长，检索成本高；卡片太短，前提、动作和来源断掉；卡片乱切，表格和步骤会变成噪声。

所以本课不是追求每个 chunk 字数差不多，而是先尊重文档结构，再在结构内部控制长度。

核心原理：chunk 是 Week8 的索引候选单元

原理	工程含义	如果做错
结构优先	先按 section_path、表格、FAQ、步骤分组	检索命中后上下文不完整
长度受控	在同一 section 内处理 token	prompt 成本和召回噪声失控
ID 稳定	chunk_id 由内容、版本、策略确定	回归测试和坏案例无法比对
元数据继承	chunk 继承 source、section、page、policy	Week8 filter 和 citation 断链
策略版本化	chunk_strategy_version 进入输出	策略变化后无法解释质量变化

参考学习时间

按课程统一节奏完成即可。

学完这一讲，你应该能做到什么

1. 解释为什么固定长度切片不适合作为主路径。
2. 设计 section-aware chunking 的基本规则。
3. 判断 overlap 什么时候有用、什么时候有害。
4. 说明 chunk_strategy_version 和稳定 chunk_id 的作用。

本课产出

- docs/blueprints/week07/chunking-strategy-v1.md
- contracts/data/document_chunk.schema.json

fixed-size vs section-aware

切片方式	优点	风险	Week7 判断
fixed-size	实现简单，长度均匀	容易切断语义、表格、步骤和标题上下文	只能作为 fallback
recursive text splitter	比固定长度更稳	仍可能忽略 parser 结构	可作为辅助
section-aware	尊重文档层级和 page/table 边界	需要更完整 metadata	Student Core 默认

切片方式	优点	风险	Week7 判断
semantic splitter	可能更贴近语义断点	依赖 embedding, 阈值和语言敏感	延后到 Week8+ 评估

场景切法决策表

场景	推荐切法	不推荐切法	原因
安装步骤	保持一个步骤组或同一标题下的连续步骤	按固定字符数切断步骤	步骤顺序本身就是语义
错误码 + 修复建议	错误码、条件、修复动作放同一 chunk	错误码和修复动作分离	Week8 命中错误码时必须带出修复证据
表格	表格整体保留；超长表格拆分时复制 header	按行随机拆或丢表头	表头决定单元格含义
FAQ	Q/A 成对保留	问题和答案分别切片	检索问题时答案必须同场出现
长段落	同 section 内 token budget 切分	跨 section overlap	既控制长度，又不污染标题层级

1. section_aware_v1 的默认规则

1. 先按 parser 输出的 section_path 分组
2. 同一 section 内合并过短 paragraph
3. 表格默认保持完整；超长表格拆分时保留 header
4. 超长 section 再按 token budget 切分
5. overlap 只在同一 section 内出现
6. 每个 chunk 记录 overlap_prev / overlap_next
7. 每个 chunk 继承 source_fingerprint、doc_version、page_no、section_path

2. table-aware 与 page-aware

表格不是普通段落。Student Core 的最低要求不是完美理解表格，而是不把表头和数据行拆成无法解释的碎片。

对象	最小处理	质量风险
短表格	保持完整 chunk	长度可能偏大，但语义完整
长表格	分段时复制 header	header 丢失会导致行含义错配
跨页表格	记录 page range 或多个 anchor	单页 citation 可能不完整
页面脚注	不混入正文 chunk	会污染检索上下文

3. chunk ID 必须稳定

禁止使用随机 UUID、时间戳或数据库自增 ID 作为 chunk 的业务 ID。

推荐公式：

```

chunk_id = sha256(
    doc_id
    + source_fingerprint
    + doc_version
    + chunk_strategy_version
    + section_id
    + chunk_index
    + content_hash
)[:32]

```

这样才能做到：

- 同一输入重复跑，ID 不变
- 改 `chunk_strategy_version`，chunk ID 变化
- 只改 `parse_run_id`，chunk ID 不变

4. overlap 的边界

Overlap 不是越大越安全。

场景	是否使用 overlap	原因
同一 section 内长段落切分	可以	保留上下文连续性
跨 section	不建议	会污染标题层级
表格	默认不用	容易重复表头和行
错误码列表	谨慎	过大 overlap 会制造重复命中
FAQ 问答	按 Q/A 保持整体	不应把问题和答案拆开

5. chunk metadata 继承规则

来源字段	chunk 是否继承	说明
<code>source_id / doc_id</code>	必须	任何 chunk 都要能回到文档
<code>source_fingerprint</code> <code>doc_version</code>	/ 必须	支撑版本复盘
<code>section_id / section_path</code>	必须	防止 chunk 失去结构上下文
<code>page_no / bbox</code>	视文档类型	PDF 必须 page, bbox 缺失要有 reason
<code>content_type / license_tag / pii_flag</code>	必须	支撑过滤和治理
<code>parse_strategy_version</code> <code>chunk_strategy_version</code>	/ 必须	支撑策略回归

继承关系可以这样理解：

层级	负责什么	下游不能丢什么
Document	source、version、fingerprint、license	文档身份和治理边界

层级	负责什么	下游不能丢什么
Section	heading path、page、bbox、section type	结构上下文和位置
Chunk	content、chunk index、overlap、quality flags	可索引证据单元
Anchor	回源位置、引用字段、缺失原因	citation 和 bad case replay

6. chunk 合格的最低标准

检查	要求
非空	content 不能为空
可回源	有 source_fingerprint / doc_version
有结构	有 section_path 或合理 missing reason
可引用	至少能生成一个 evidence anchor
可回归	有 chunk_strategy_version
可过滤	有 content_type / license_tag / pii_flag

7. 坏案例

坏案例	结果	修复
表头和数据行被拆开	检索命中一行数字但不知道字段含义	表格 chunk 保留 header
操作步骤被拆断	Agent 答案漏前置条件	同一 section 内合并短步骤
错误码和解决方案分离	命中错误码但无法给出修复动作	错误码组整体切片
overlap 跨 section	引入上一个主题的无关上下文	overlap 只允许同 section

企业最佳实践

实践	说明
对 FAQ 保持问答成对	用户问题命中 FAQ 时，答案必须同场出现
对错误码表保留 header	错误码、症状、修复动作的列含义不能断
对安装步骤保留前置条件	缺少前置条件的步骤容易误导操作
对超长 section 先切逻辑块，再切 token	不要一上来用固定字符数
把 bad chunk 写入 review	策略调优要基于坏案例，不是凭感觉

课堂练习

给一段文档结构：

```
H2: Reset Edge Gateway
P: Before reset, confirm firmware version.
Table: error_code | symptom | action
P: After reset, wait for provisioning heartbeat.
```

请设计 chunk 边界。合格答案应说明：表格是否独立、前置条件是否和步骤同 chunk、chunk ID 如何稳定、anchor 如何覆盖 page/bbox。

本课最重要判断

切片策略不是“为了凑 embedding 输入长度”，而是为了让 Week8 能检索到 **完整、可引用、可质检** 的证据单元。

自检清单

- 我知道 fixed-size chunk 的风险。
- 我能写出 `section_aware_v1` 的基本规则。
- 我知道 overlap 不能跨 section 滥用。
- 我能解释稳定 chunk ID 为什么重要。

最小行动命令

```
# 可执行：已与主项目 Week7 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
  python -m pipelines.parse_normalize.run_parse \
  --manifest-path data/seed_manifests/manifest_workspace_helpcenter_v1.json \
  --parser auto \
  --chunk-strategy section_aware_v1 \
  --data-release-id week07-lesson03-chunking \
  --dry-run \
  --artifacts-dir artifacts/week07 \
  --report-json reports/week07/parse_run_report.json
```

延伸阅读

- LangChain recursive text splitter: https://docs.langchain.com/oss/python/integrations/splitters/recursive_text_splitter。理解递归切分的基本思路，但不要把它当成唯一策略。
- Docling chunking 文档: <https://docling-project.github.io/docling/concepts/chunking/>。对照 Docling 的结构化 chunking 能力。
- LlamaIndex node parser / text splitters: https://docs.llamaindex.ai/en/stable/module_guides/loading/node_parsers/。作为扩展阅读，理解不同框架对 chunk 边界的抽象。

Footnotes

1. fixed-size chunk 可以作为 fallback，但不应该成为 Week7 Student Core 的主路径。
2. overlap 只解决局部上下文断裂，不能弥补错误 section 边界。
3. stable chunk ID 的目标不是“更漂亮”，而是让 parse / chunk strategy 变化可以被回归比较。
4. 本课不做 semantic splitter 的生产化，因为语义切分通常依赖 embedding 或模型阈值，应放到 Week8+ 评估。